

Running Native Extensions and Ruby Code in Parallel Threads

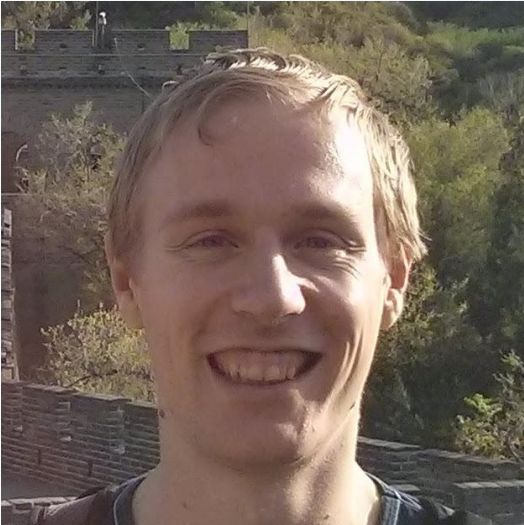
Benoit Daloze

EURUKO 2026 **B** | **R** | **N** | **O** |



DATADOG

Who am I?



Benoit Daloze

GitHub: @eregon

Website: <https://eregon.me>

Bluesky: @eregon.me

Mastodon:
@eregon@ruby.social

PhD on parallelism in dynamic languages

Maintainer of:

- TruffleRuby
- ruby/setup-ruby
- ruby/spec

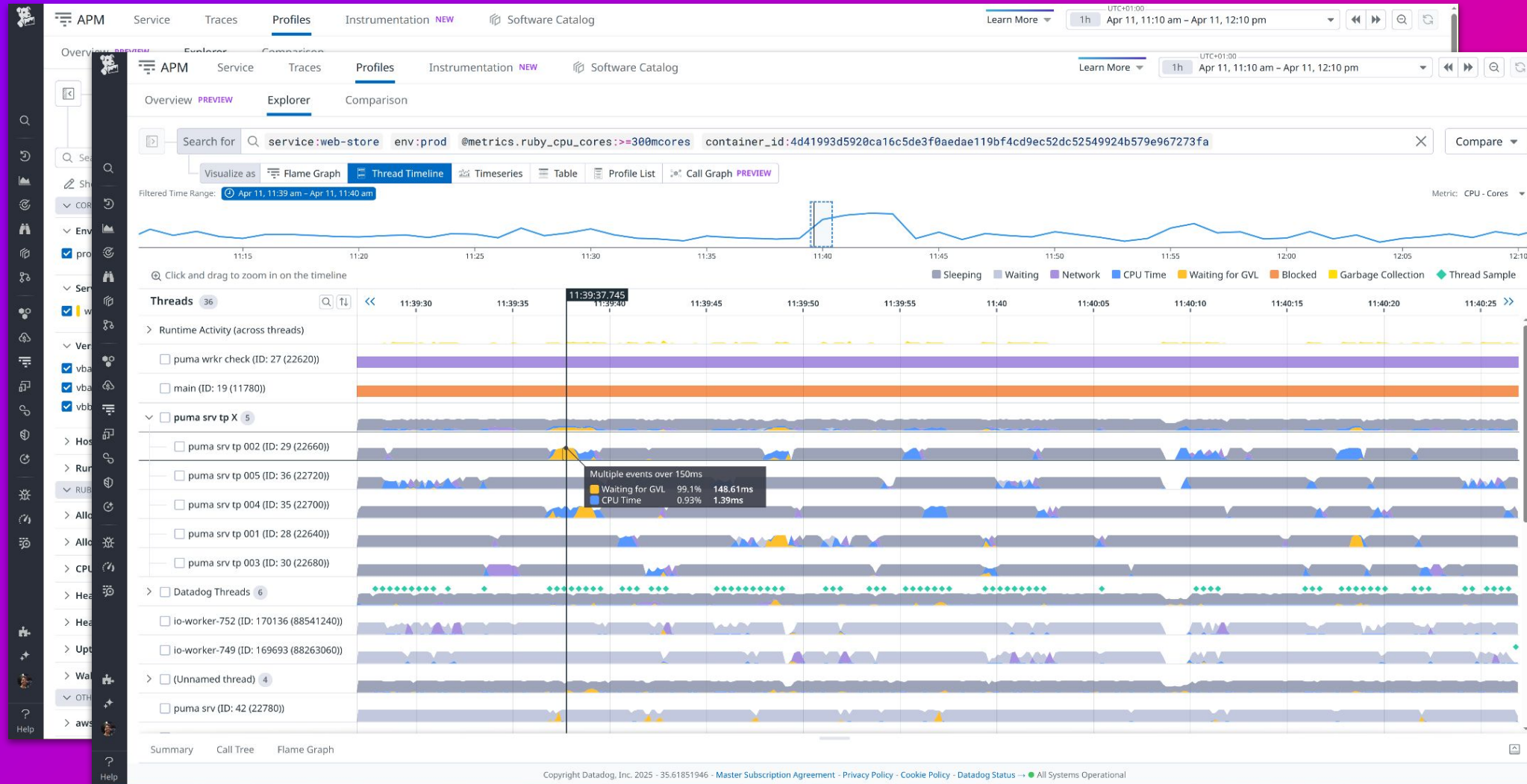
CRuby committer

Now working at Datadog on the Ruby Continuous Profiler!



Modern monitoring and security as a Service

Including profiling, distributed tracing, test optimization, security, errors, logs for Ruby

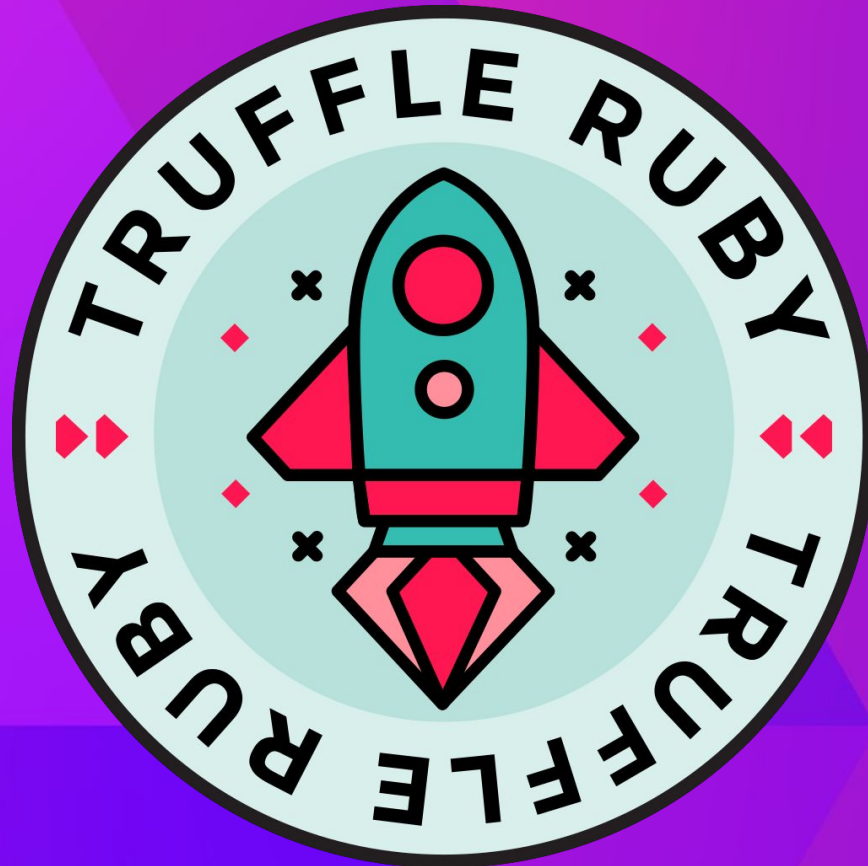


What if we could?

- Run Ruby code in parallel
- Run native extensions in parallel
- Forget about the Global VM Lock
- Be faster even on a single thread
- And still choose the programming model we want?

What if we could?

We can do all that,
today, with



TruffleRuby

- Is a high-performance Ruby implementation
- Runs Ruby code in parallel
- Runs native extensions in parallel if marked Thread-safe
- Does not have a Global VM Lock
- Uses Threads for parallelism, the most general model, so you can pick shared memory, actors, futures, STM, etc

Agenda

- Demo
- Programming models
- How C extensions are implemented
- How to make C extensions thread-safe

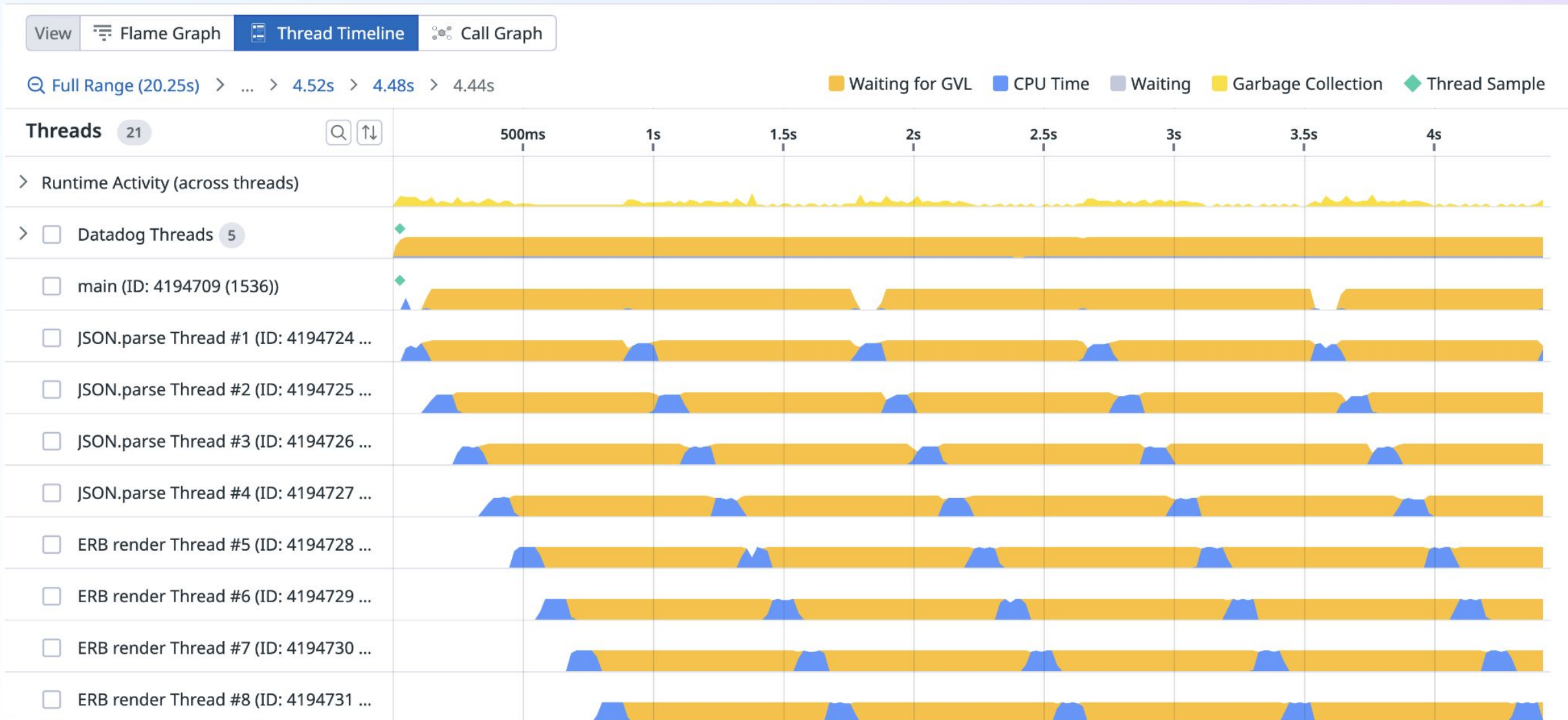
Demo Time!

4 threads × 1000 JSON.parse of a 20 MB document (C extension)
4 threads × 4000 ERB renders of a 5000-row HTML report (pure Ruby)

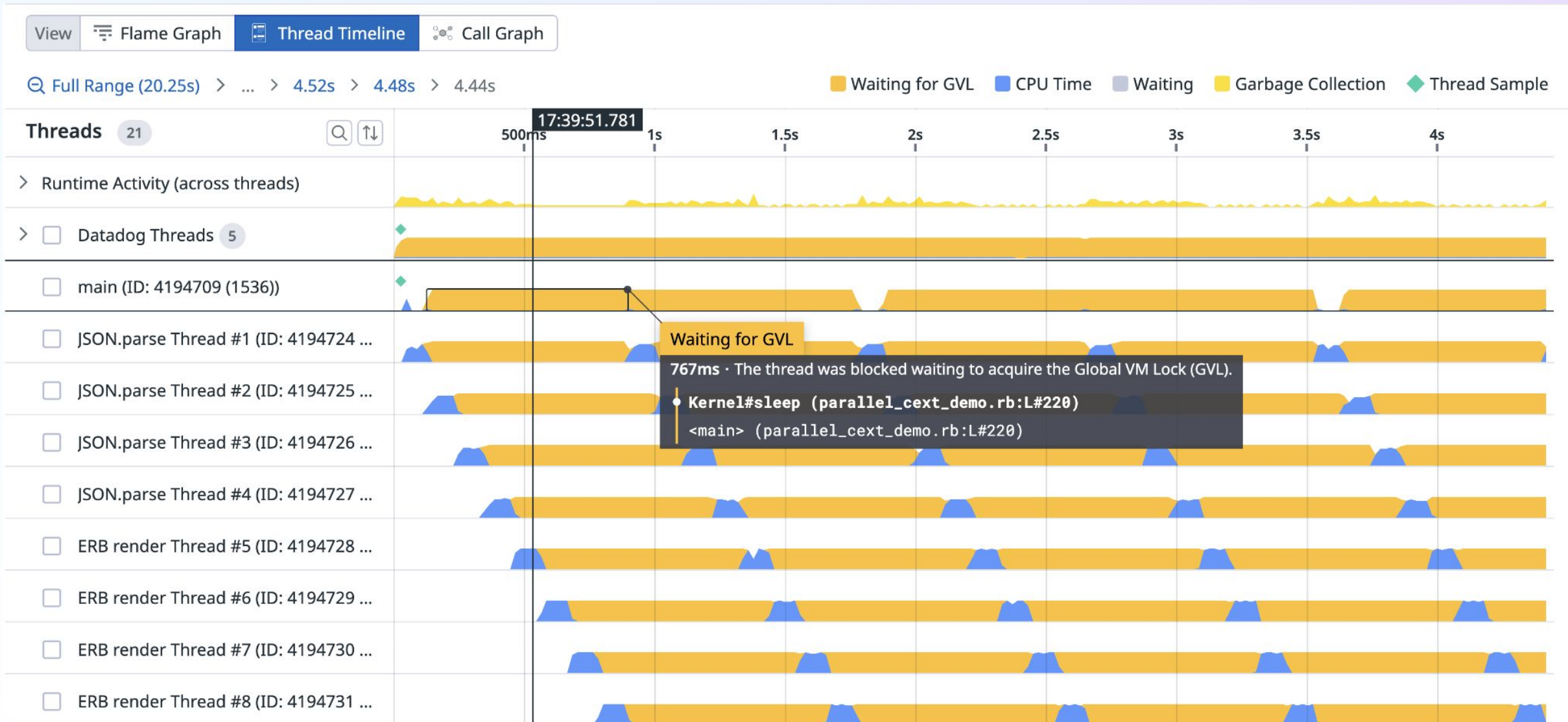
C ext (JSON.parse)	#1		797/1000
C ext (JSON.parse)	#2		801/1000
C ext (JSON.parse)	#3		792/1000
C ext (JSON.parse)	#4		801/1000
Ruby (ERB render)	#5		2772/4000
Ruby (ERB render)	#6		2831/4000
Ruby (ERB render)	#7		2859/4000
Ruby (ERB render)	#8		2849/4000

elapsed: 14.9s

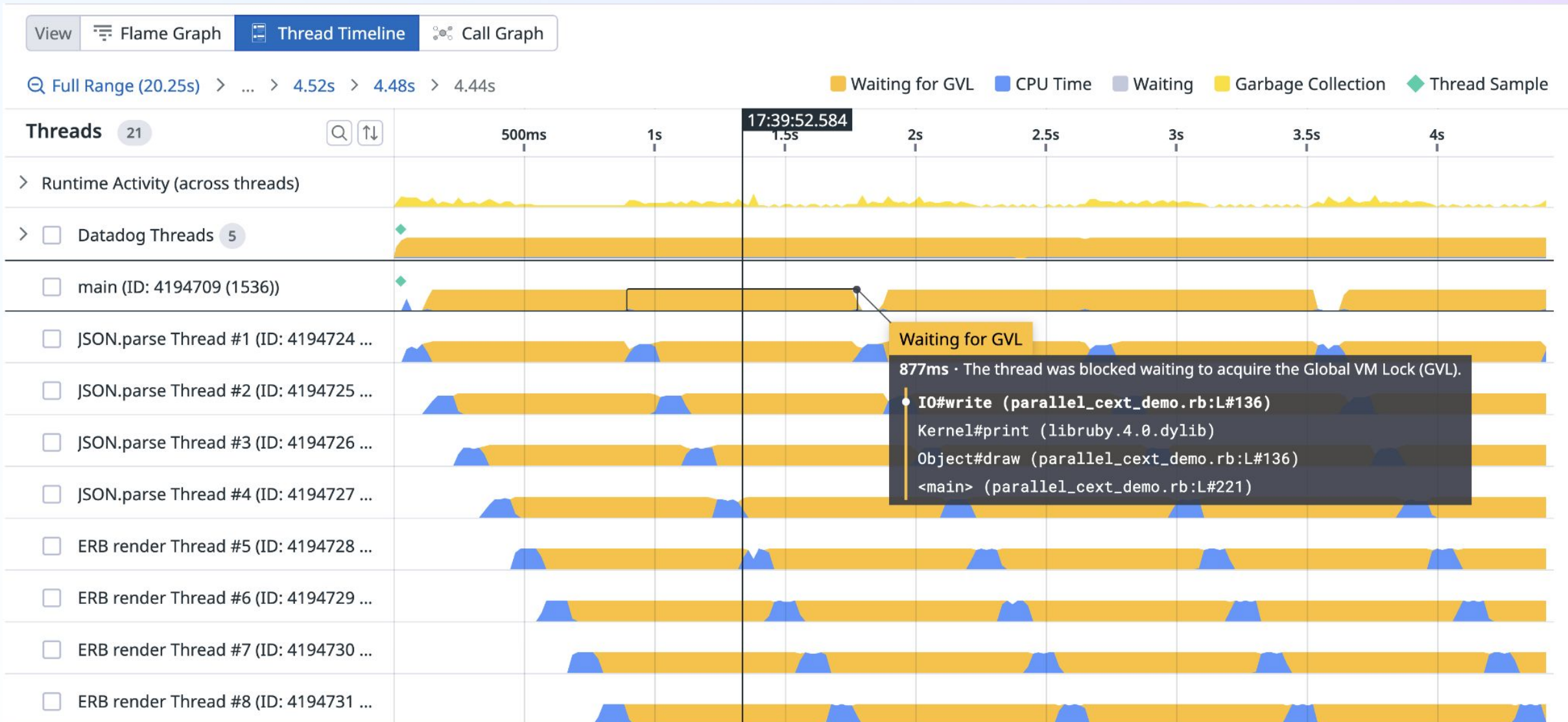
Seeing the Global VM Lock



Seeing the Global VM Lock



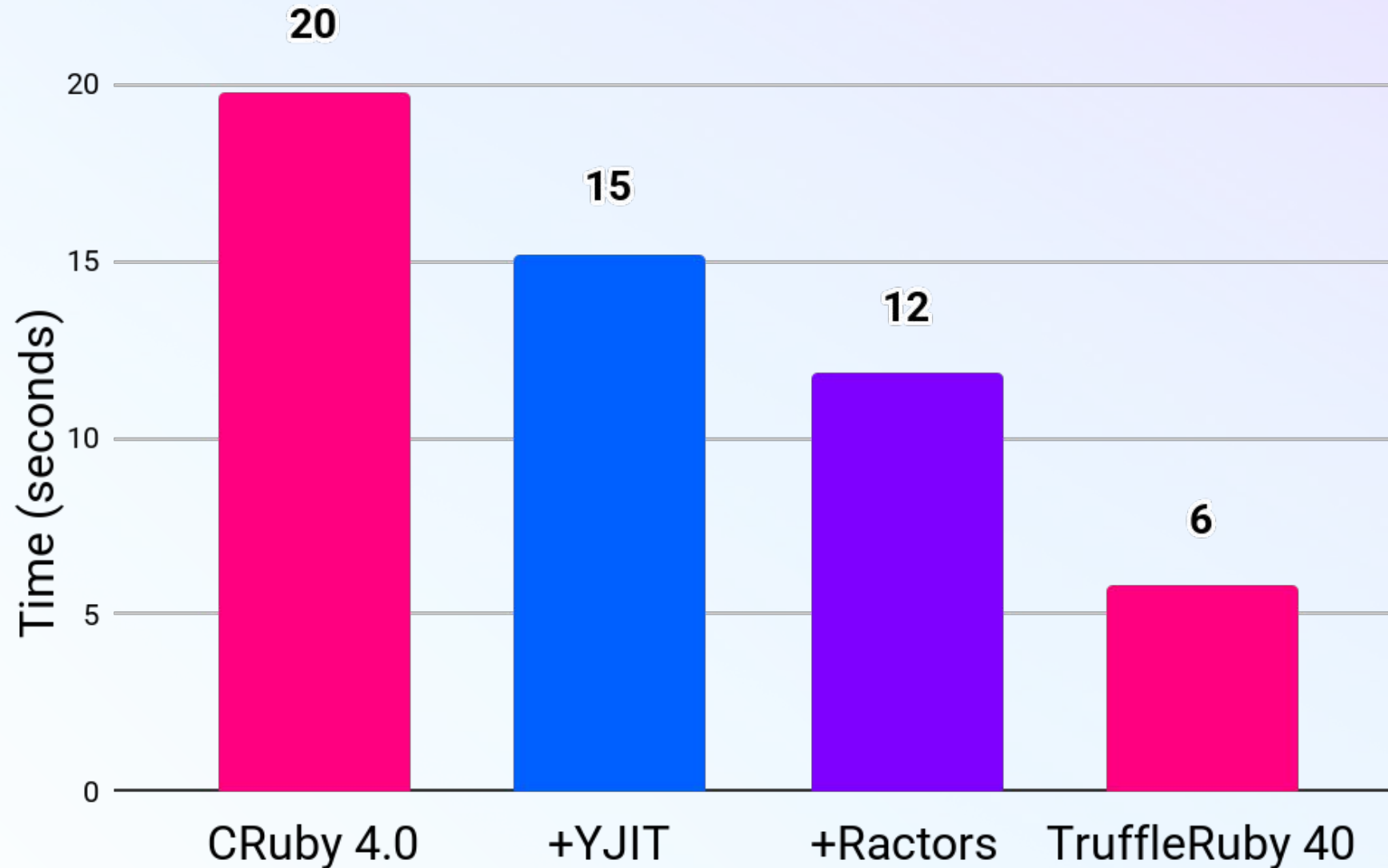
Seeing the Global VM Lock



What about Ractors?

- Ractors is a way to run code in parallel in CRuby, using the actor model
- Runs Ruby code in parallel only if the code is Ractor-compatible
(no shared state allowed)
- Runs native extensions in parallel if marked Ractor-safe
- Still a Global VM Lock per Ractor, and a process-wide lock
- Forces to use the Actor programming model:
 - more restrictive
 - extra copying and overhead

Demo Results (lower is better)



Agenda

- Demo
- Programming models
- How C extensions are implemented
- How to make C extensions thread-safe

Does the programming model matter?

```
JOB = [Job.new(:cext, CEXT_ITERS, -> { parse_json })] * 4 +  
      [Job.new(:ruby, RUBY_ITERS, -> { render_page })] * 4
```

```
progress = Array.new(JOB.size, 0)  
threads = JOB.map.with_index do |job, i|  
  Thread.new do  
    job.its.times do  
      job.run_once  
      progress[i] += 1  
    end  
  end  
end
```

```
until threads.all?(&:stop?)  
  sleep 0.1  
  draw(progress)  
end  
threads.each(&:join)
```

Does the programming model matter?

```
progress = Array.new(JOBS.size, 0)
threads = JOBS.map.with_index do |job, i|
  Thread.new do
    job.itsers.times do
      job.run_once
      progress[i] += 1
    end
  end
end

until threads.all?(&:stop?)
  sleep 0.1
  draw(progress)
end
threads.each(&:join)
```

```
progress = Array.new(JOBS.size, 0)
port = Ractor::Port.new
ractors = JOBS.map.with_index do |job, i|
  Ractor.new(port, job, i) do |port, job, i|
    job.itsers.times do
      job.run_once
      port << i
    end
  end
end

remaining = JOBS.sum(&:itsers)
last_draw = START
until remaining == 0
  i = port.receive
  progress[i] += 1
  remaining -= 1
  now = monotonic_time
  if now - last_draw >= 0.1
    draw(progress)
    last_draw = now
  end
end
ractors.each(&:join)
```

Does the programming model matter?

Ractor needs more changes:

- Cannot just pass a block/Proc to a Ractor, need:

```
block = Ractor.shareable_proc(&block) if RACTORS
```

- Cannot use constants without making their value immutable:

```
Ractor.make_shareable(ARTICLES) if RACTORS
```

A key point: Compatibility

- Most Ruby gems work fine in Threads
- Most Ruby gems do not work in a Ractor
 - No popular test harness can run in a Ractor yet: minitest, rspec, test-unit
 - Running tests in parallel requires Threads or Processes
 - Anything with shared state is hard to migrate to Ractors
- Huge effort to make Ruby gems compatible with Ractor
- Overfit: Ractors are CRuby's current form of in-process parallelism, but:
- Not the only way: TruffleRuby, JRuby and CPython have parallel threads
- matz/spinel, a compiler from Ruby to C, has parallel threads too!

Spinel

- An AOT compiler from a subset of Ruby to C
- Fast
- Has parallel threads
- Supports many Ruby features and a few standard libraries
- But not compatible with gems
- And no dynamic `#require`, no `#eval`, no dynamic `#instance_variable_set`
- A much more static subset of Ruby

Advantages of Ractors

- State is isolated per Ractor, fails early if it does not hold
- Guaranteed sequential access to state and no data races, when not mixed with Threads
- Race conditions are different: instead of “which threads can access this?” it’s “what if messages arrive in a different order?”

Indications that the Actor model is not enough

There are several gems adding shared state to Ractor:

- `ratomic`: shared counter, concurrent map, etc
- `atomic-ruby`: atomic variables and `CountDownLatch`
- `ractor-sharing`: transactional memory, concurrent Hash, etc

All of these break the guarantees of the actor model:

- State no longer isolated per actor, adds data races and race conditions
- They use `RUBY_TYPED_FROZEN_SHAREABLE` incorrectly:
“the mutation of wrapped data is not allowed.”

Agenda

- Demo
- Programming models
- How C extensions are implemented
- How to make C extensions thread-safe

How C Extensions are Implemented in TruffleRuby

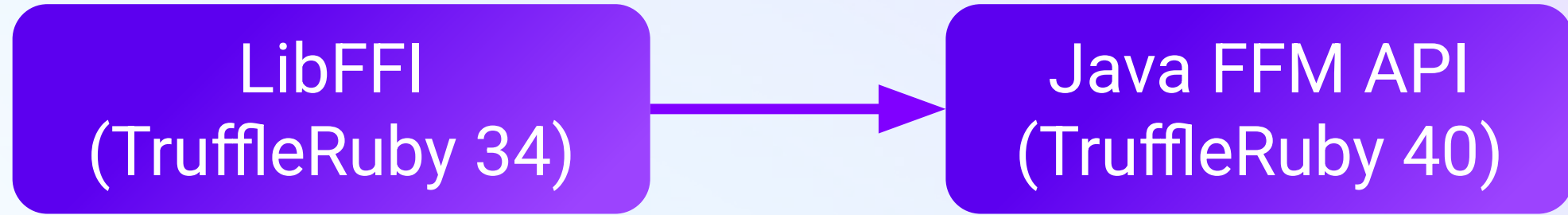
TruffleRuby originally used Sulong, an interpreter + JIT for C code

- That allowed to store Ruby/Java objects inside C variables
- Cool technology, great peak performance, bad startup, terrible warmup
- Not a great trade-off considering TruffleRuby is noticeably slow at warmup

Now C extensions are compiled by gcc/clang and run natively, like on CRuby

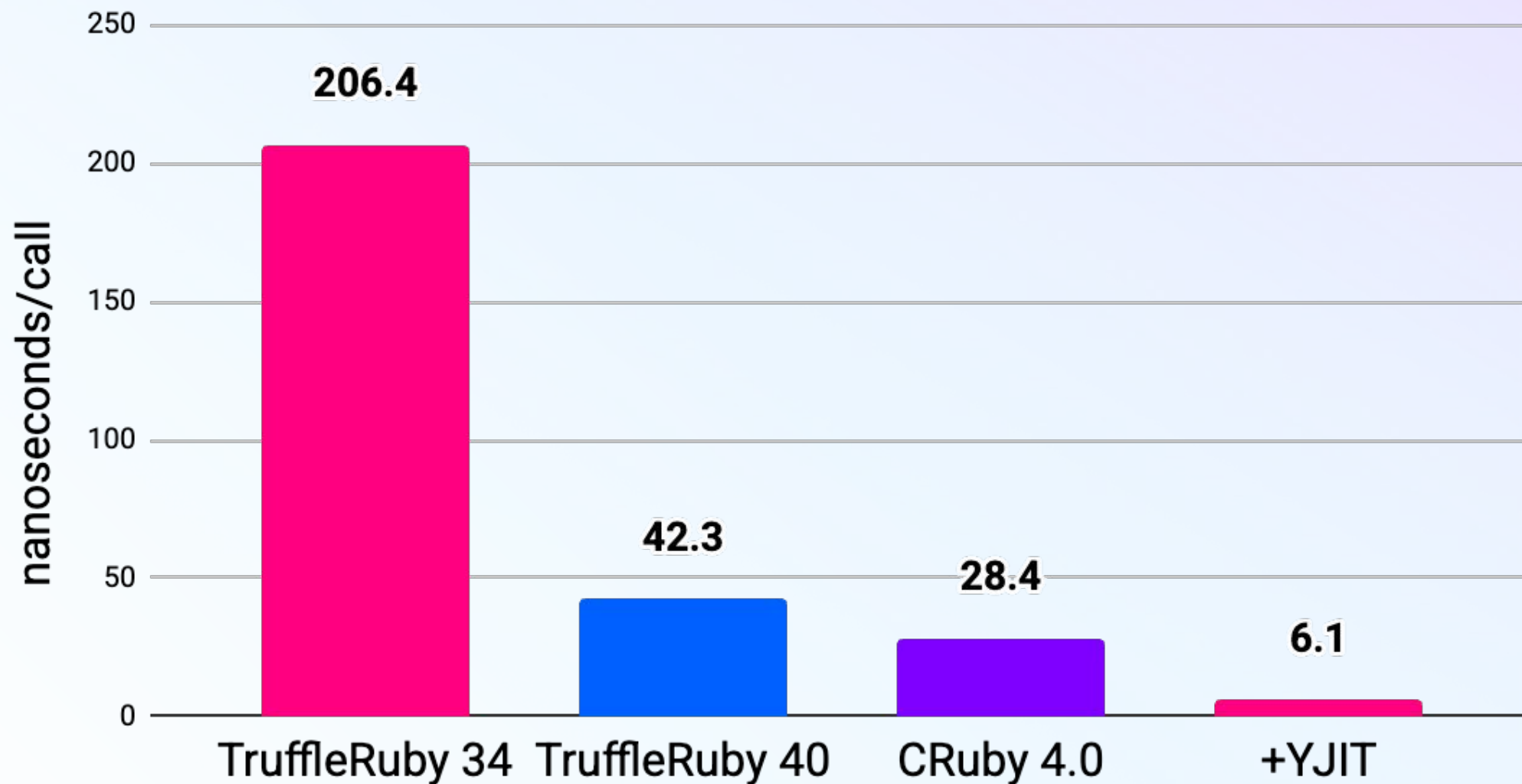
- Performance of calls from Java to C and C to Java becomes crucial
- We cannot store Ruby/Java objects inside C variables, we need *handles*

Calling C and being called by C

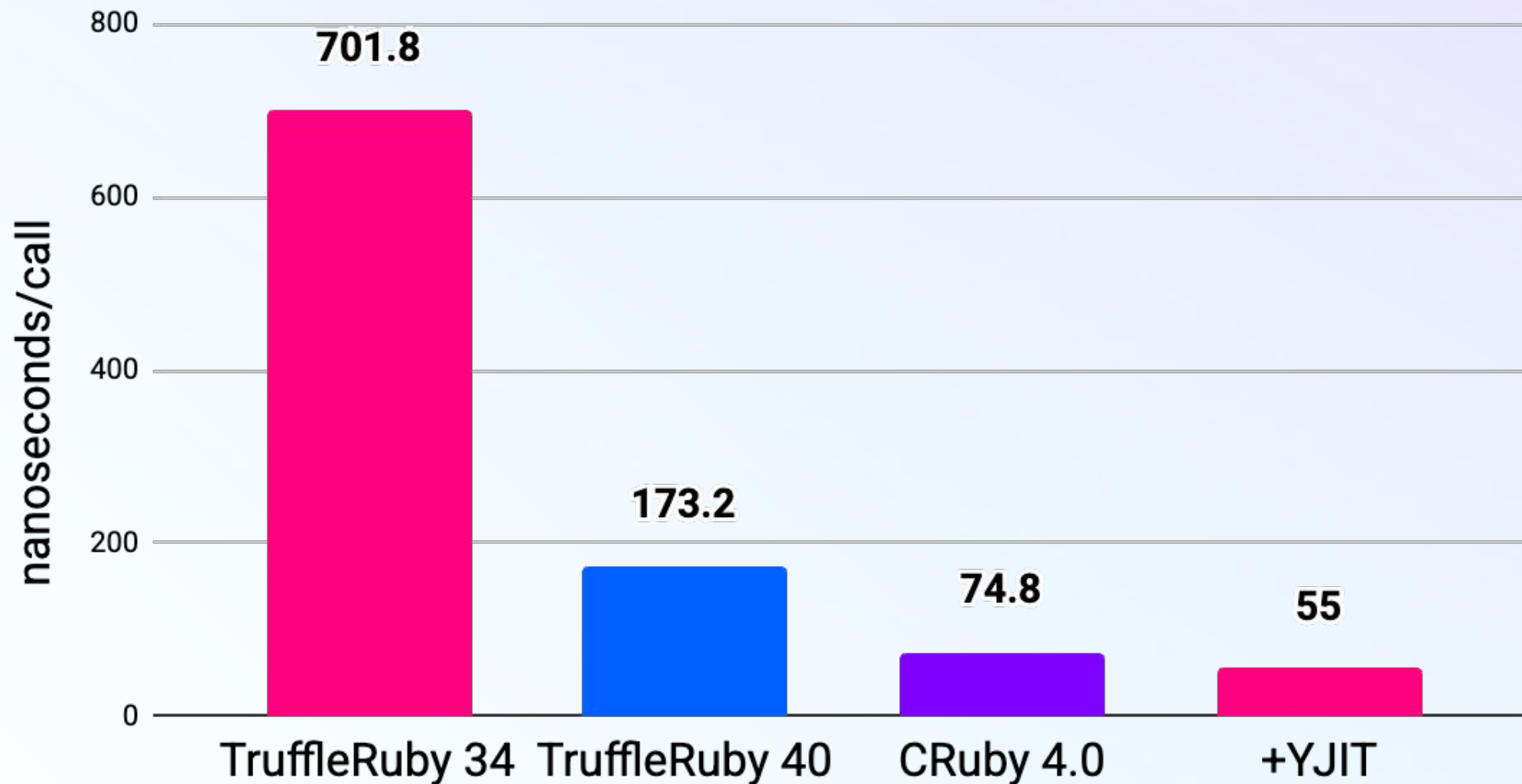


- Generic way to reflectively call a C function (e.g. from Ruby) and to create a C function to go back from C to Ruby
 - Quite slow, generic argument handling, like an interpreter interpreting a *function signature* to call a function
 - Extra layer in Java: use JNI to call a known C function, and then use LibFFI to call an arbitrary C function
- Java→C: knows the C ABI to call C functions, generate the code to move arguments in the right registers
 - C→Java: *generates* a C function, specialized for the argument types
 - Like a JIT compiler optimizing a specific *function signature*

Downcall: Ruby → C



Downcall + Upcall: Ruby → C → Ruby



C Extensions Handles

20 bits

prefix:
0x0bade

29 bits

block index:
1

12 bits

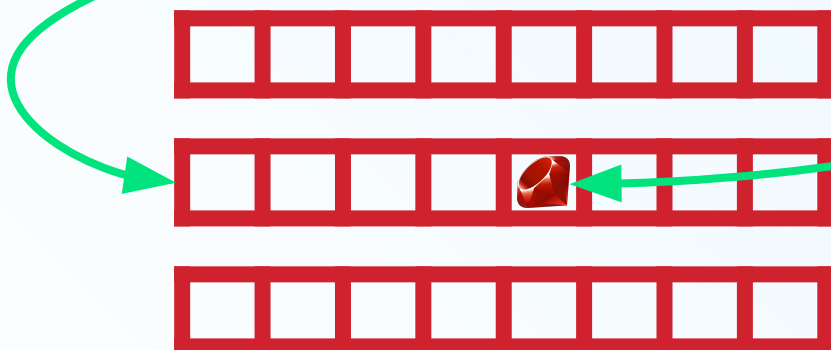
offset in block:
4

3 bits

suffix:
000

Native memory

Managed memory



On 64-bit machines,
only the low 48 bits are addressable,
so handles cannot point to valid memory

Implementing C Extensions with Ruby Code

C: `VALUE (*upcall_rb_ary_push)(VALUE array, VALUE element);`

```
/** Appends one element at the end of the array */  
VALUE rb_ary_push(VALUE array, VALUE element) {  
    return upcall_rb_ary_push(array, element);  
}
```

Java: `public static long upcall_rb_ary_push(long array, long element) {
 return runtime.upcall(INDEX_rb_ary_push, array, element);
}`

Ruby: `def rb_ary_push(array, element)
 array << element
end`

Agenda

- Demo
- Programming models
- How C extensions are implemented
- How to make C extensions thread-safe

Making the C API Thread-safe in TruffleRuby

- Since the Ruby C API, `rb_*`, is implemented basically with Ruby code
- And Ruby code in TruffleRuby already runs in parallel without a Global VM Lock
- Then effectively (most of) the Ruby C API is thread-safe already!

We still need some adjustments in our internals:

- Synchronization around creating handles
- Thread-safe weak references and cleanup of handle blocks

Making an extension Thread-safe (1)

- Document which objects are safe to share between threads and which are not
- This already needs to be done on CRuby, as threads run concurrently, and extensions can release the GVL on many `rb_*` calls

```
# JSON::Coder instances are thread-safe  
# and can be used from multiple threads.
```

```
class JSON::Coder  
end
```

```
# JSON::Ext::Generator::State instances are NOT thread-safe  
# and must only be used by a single Thread.
```

```
class JSON::Ext::Generator::State  
end
```

Making an extension Thread-safe (2)

- The extension's own code must be thread-safe, e.g. not access/mutate shared state without synchronization:

```
struct buffer {  
    char* ptr;  
    int size;  
    int capacity;  
};  
  
void append(struct buffer* buf, char c) {  
    if (buf->size == buf->capacity) {  
        // realloc()  
    }  
    buf->ptr[buf->size++] = c;  
}
```

Making an extension Thread-safe (3)

- When calling external library functions (e.g. from libc), ensure they are thread-safe or that you use sufficient synchronization

```
void thread1() {  
    chdir("/tmp");  
}
```

```
void thread2() {  
    open("README.md", O_RDONLY);  
}
```

How to mark an extensions as Thread-safe?

```
void Init_my_extension(void) {  
    #ifdef HAVE_RB_EXT_THREAD_SAFE  
    rb_ext_thread_safe(true);  
    #endif
```

```
    // The C function foo_impl can be called from multiple Threads in parallel  
    rb_define_method(myClass, "foo", foo_impl, 0);  
}
```

Making an extension Ractor-safe

- Making C extensions Thread-safe and Ractor-safe is fairly similar
 - The same first 3 requirements
 - 2 more requirements for Ractors

Making an extension Ractor-safe (4)

- Unshareable objects must not be exposed to another Ractor (otherwise crash)

```
VALUE global_cache;
```

```
VALUE get_cache(VALUE klass) {  
    return global_cache;  
}
```

In a non-main Ractor:
Foo.cache # => crash

```
void Init_foo() {  
    VALUE cFoo = rb_define_class("Foo", rb_cObject);  
    global_cache = rb_hash_new();  
    rb_define_singleton_method(cFoo, "cache", get_cache, 0);  
}
```

Making an extension Ractor-safe (5)

- Optionally, mark objects that can be accessed safely by multiple Ractors as shareable via `RUBY_TYPED_FROZEN_SHAREABLE`
- This means the object must not allow any mutation after it is frozen

```
static const rb_data_type_t BigDecimal_data_type = {  
    "BigDecimal",  
    { 0, BigDecimal_delete, BigDecimal_memsize, },  
    0,  
    0,  
    RUBY_TYPED_FROZEN_SHAREABLE |  
    RUBY_TYPED_FREE_IMMEDIATELY | RUBY_TYPED_WB_PROTECTED  
};
```

How to mark an extensions as Ractor-safe?

```
void Init_my_extension(void) {  
    #ifdef HAVE_RB_EXT_RACTOR_SAFE  
    rb_ext_ractor_safe(true);  
    #endif  
  
    // The C function foo_impl can be called from multiple Ractors in parallel  
    rb_define_method(myClass, "foo", foo_impl, 0);  
}
```

Note: Ractor-safe implies Thread-safe

Making code Ractor-safe

- Making C extensions Thread-safe and Ractor-safe is fairly similar
 - The same first 3 requirements
 - 2 more requirements for Ractors
- Making Ruby code compatible with Ractors is harder
 - Cannot use constants unless their value are immutable
 - Cannot use class/module `@ivar` unless their value are immutable
 - Cannot use shared state like global caches, etc

Conclusion

There are multiple ways to run Ruby code and native extensions in parallel:

- Parallel Threads on TruffleRuby
- Parallel Ractors on CRuby
- Multiple processes

Those have different programming models:

- Threads → shared memory, the most general, can use anything on top
- Ractors → share only immutable data structures
- Multiple processes → share nothing (except copy-on-write memory with `#fork`)

CRuby might support more than Ractors in the future:

- TruffleRuby, JRuby, Spinel and CPython already support parallel threads
- Most gems work with Threads and don't work with Ractors

Making Hash Parallel, Thread-Safe and Fast!



Benoit Daloze



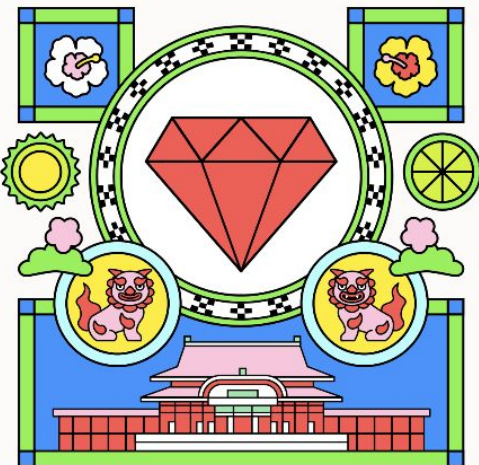
RubyKaigi 2026



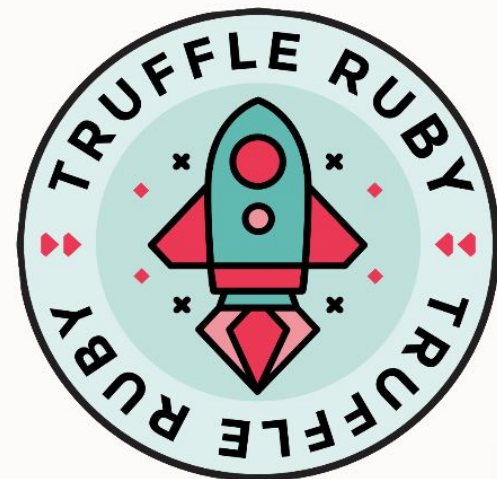
DATADOG

From Interpreting C Extensions to Compiling Them

Benoit Daloze



RubyKaigi
2024  **MAY15-17**
@NAHA, OKINAWA



Give TruffleRuby 40 a try

Install TruffleRuby in a few seconds! (it's always precompiled)

```
$ mise install ruby@truffleruby-40.0.0
```

Try TruffleRuby for your application, Ruby program, etc

Test TruffleRuby in CI

- `uses: ruby/setup-ruby@v1`
 `with:`
 `ruby-version: truffleruby`

Contribute to TruffleRuby

Fully Open-Source, you can just make a PR on GitHub:

<https://github.com/truffleruby/truffleruby>

Any Question?

Running Native Extensions and Ruby Code in Parallel Threads

```
truffleruby 40.0.0-dev-596fde49 Native
4 threads x 1000 JSON.parse of a 20 MB document (C extension)
4 threads x 4000 ERB renders of a 5000-row HTML report (pure Ruby)

C ext (JSON.parse) #1 1000/1000
C ext (JSON.parse) #2 1000/1000
C ext (JSON.parse) #3 1000/1000
C ext (JSON.parse) #4 1000/1000
Ruby (ERB render) #5 4000/4000
Ruby (ERB render) #6 4000/4000
Ruby (ERB render) #7 4000/4000
Ruby (ERB render) #8 4000/4000
elapsed: 6.0s
6.5 cores busy
```

Benoit Daloze

EURUKO 2026 **B** | **R** | **N** | **O**



DATADOG